

Task 1

1. What is Spark Structured Streaming?

Normally, with Spark batch:

```
JSON file
  ↓
Spark
  ↓
DataFrame
  ↓
Transform
  ↓
Output
```

With Structured Streaming:

```
New JSON files arrive
  ↓
Spark Streaming
  ↓
DataFrame
  ↓
Transform
  ↓
Output
  ↓
Keeps watching for new files
```

The important idea is:

Streaming means Spark keeps processing new data as it arrives.

2. Our beginner project

Let's imagine this folder:

```
/data/streaming/input/
```

New JSON files will arrive there.

For example:

```
order1.json
```

```
order2.json
order3.json
```

Each JSON file contains an order:

```
{
  "order_id": "ORD1001",
  "timestamp": "2026-08-27T10:15:00Z",
  "customer": {
    "customer_id": 501,
    "name": "John Doe",
    "email": "john@example.com",
    "address": {
      "city": "Toronto",
      "postal_code": "M5V"
    }
  },
  "product": "Laptop",
  "category": "Electronics",
  "quantity": 1,
  "price": 1200.0
}
```

We want to transform it into something simpler:

```
order_id
customer_id
customer_name
city
product
category
quantity
price
total_amount
timestamp
```

Where:

```
total_amount = quantity × price
```

Then save the result as **Parquet**.

3. Step 1 — Create SparkSession

First, import Spark.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
```

```
spark = (  
    SparkSession.builder  
        .appName("JSON Streaming")  
        .getOrCreate()  
)  
  
print("Spark version:", spark.version)
```

What is `spark`?

`SparkSession` is our main entry point to Spark.

We use:

`spark`

to read data, create `DataFrames`, and start streaming queries.

4. Step 2 — Read JSON as a stream

This is the most important difference.

Batch:

```
df = spark.read.json("hdfs://localhost:9000/data/streaming/input")
```

Streaming:

```
df = spark.readStream.json(  
    "hdfs://localhost:9000/data/streaming/input"  
)
```

Notice:

`read`

became:

`readStream`

So:

```
df = spark.readStream.json(  
    "hdfs://localhost:9000/data/streaming/input"  
)
```

means:

Spark continuously watches this directory for new JSON files.

5. Step 3 — Understand the streaming DataFrame

Now we have:

```
df
```

It is a **streaming DataFrame**.

You can think of it as:

```
df
↓
new JSON files
↓
new rows
↓
new rows
↓
new rows
```

But the query hasn't started yet.

This is important.

When we write:

```
df = spark.readStream.json(...)
```

Spark does **not** start processing data yet.

6. Step 4 — Define the schema

For beginners, I recommend explicitly defining the JSON schema.

```
schema = StructType([
    StructField("order_id", StringType()),
    StructField("timestamp", StringType()),
```

```

StructField(
  "customer",
  StructType([
    StructField("customer_id", IntegerType()),
    StructField("name", StringType()),
    StructField("email", StringType()),
    StructField(
      "address",
      StructType([
        StructField("city", StringType()),
        StructField("postal_code", StringType())
      ])
    )
  ])
),
StructField("product", StringType()),
StructField("category", StringType()),
StructField("quantity", IntegerType()),
StructField("price", DoubleType())
])

```

Then:

```

df = spark.readStream.schema(schema).json(
  "hdfs://localhost:9000/data/streaming/input"
)

```

Now Spark knows the structure of the incoming JSON.

7. Step 5 — Transform the data

Our JSON has nested data:

```

customer
├── customer_id
├── name
├── email
└── address
    ├── city
    └── postal_code

```

We can extract those fields.

```

transformed_df = df.select(
  col("order_id"),
  col("customer.customer_id").alias("customer_id"),
  col("customer.name").alias("customer_name"),
  col("customer.address.city").alias("city"),
  col("product"),
  col("category"),

```

```

    col("quantity"),
    col("price"),
    (col("quantity") * col("price")).alias("total_amount"),
    col("timestamp")
)

```

Now our DataFrame looks conceptually like:

```

+-----+-----+-----+-----+-----+-----+-----+
|order_id|customer_id|customer_name|city|product|category|
|quantity|price|total_amount|timestamp|
+-----+-----+-----+-----+-----+-----+-----+
|ORD1001|501|John Doe|Toronto|Laptop|Electronics|1
|1200.0|1200.0|2026-08-27...|
+-----+-----+-----+-----+-----+-----+-----+

```

8. Step 6 — Write the streaming output

This is where we actually start the streaming query.

For example, save to Parquet:

```

query = (
    transformed_df.writeStream
        .format("parquet")
        .outputMode("append")
        .option("path", "hdfs://localhost:9000/data/streaming/output")
        .option("checkpointLocation",
            "hdfs://localhost:9000/data/streaming/checkpoint")
        .start()
)

```

There are several important parts here.

.writeStream

Tells Spark:

I want to write streaming data.

.format("parquet")

Output format:

Parquet

```
.outputMode("append")
```

New records are appended to the output.

```
.option("path", ...)
```

Where should the output go?

```
/data/streaming/output
```

```
.option("checkpointLocation", ...)
```

Spark stores streaming progress information here.

```
/data/streaming/checkpoint
```

The checkpoint is **very important** in Structured Streaming.

```
.start()
```

This actually starts the streaming query.

9. Step 7 — Keep the streaming application running

After:

```
.start()
```

Spark starts the streaming query.

But if your Python program immediately finishes, the application can stop.

So we use:

```
query.awaitTermination()
```

This means:

Keep the streaming application running and wait for the query to terminate.

So the complete flow is:

```
readStream
```

```
    ↓
transform
    ↓
writeStream
    ↓
start()
    ↓
awaitTermination()
```

10. Complete beginner example

Here is the complete code:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

# =====
# 1. Create Spark Session
# =====

spark = (
    SparkSession.builder
        .appName("JSON Streaming")
        .getOrCreate()
)

print("Spark version:", spark.version)

# =====
# 2. Define JSON schema
# =====

schema = StructType([
    StructField("order_id", StringType()),
    StructField("timestamp", StringType()),

    StructField(
        "customer",
        StructType([
            StructField("customer_id", IntegerType()),
            StructField("name", StringType()),
            StructField("email", StringType()),

            StructField(
                "address",
                StructType([
                    StructField("city", StringType()),
                    StructField("postal_code", StringType())
                ])
            )
        ])
    ])
])
```

```

        ),

        StructField("product", StringType()),
        StructField("category", StringType()),
        StructField("quantity", IntegerType()),
        StructField("price", DoubleType())
    ])

# =====
# 3. Read JSON as Streaming
# =====

df = (
    spark.readStream
        .schema(schema)
        .json("hdfs://localhost:9000/data/streaming/input")
)

# =====
# 4. Transform Data
# =====

transformed_df = df.select(
    col("order_id"),
    col("customer.customer_id").alias("customer_id"),
    col("customer.name").alias("customer_name"),
    col("customer.address.city").alias("city"),
    col("product"),
    col("category"),
    col("quantity"),
    col("price"),
    (col("quantity") * col("price")).alias("total_amount"),
    col("timestamp")
)

# =====
# 5. Write Streaming Output
# =====

query = (
    transformed_df.writeStream
        .format("parquet")
        .outputMode("append")
        .option(
            "path",
            "hdfs://localhost:9000/data/streaming/output"
        )
        .option(
            "checkpointLocation",
            "hdfs://localhost:9000/data/streaming/checkpoint"
        )
        .start()
)

```

```
# =====  
# 6. Keep Streaming Application Running  
# =====  
  
query.awaitTermination()
```

11. What happens when we run this?

Suppose initially:

```
/data/streaming/input/
```

is empty.

You start the Python program:

```
Spark  
↓  
Watching input directory
```

Then you put:

```
order1.json
```

into the input directory.

Spark detects it:

```
order1.json  
↓  
readStream  
↓  
DataFrame  
↓  
transform  
↓  
Parquet
```

Then another file arrives:

```
order2.json
```

Spark processes it too:

```
order2.json  
↓  
readStream  
↓
```

```
transform
  ↓
Parquet
```

And it keeps doing this.

12. The most important concepts to remember

For now, focus on these **7 things**:

Concept	Meaning
<code>readStream</code>	Read data continuously
Streaming DataFrame	DataFrame representing incoming data
Transformation	Modify/process the data
<code>writeStream</code>	Write streaming results
<code>outputMode</code>	How results are written
<code>checkpointLocation</code>	Stores streaming progress/state
<code>awaitTermination()</code>	Keeps application running

The basic Structured Streaming pattern is:

```
df = spark.readStream...
```

↓

```
transformed_df = df.select(...)
```

↓

```
query = transformed_df.writeStream...
```

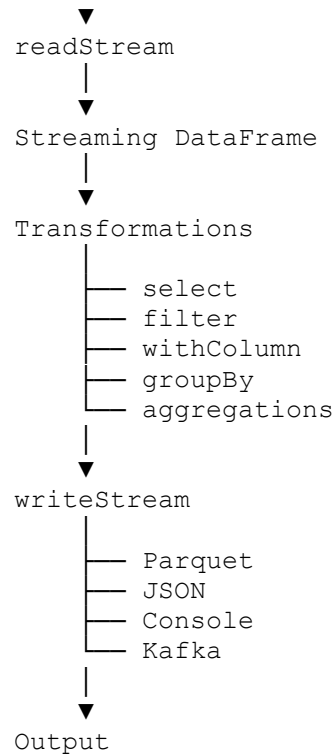
↓

```
query.awaitTermination()
```

Your mental model

SPARK STRUCTURED STREAMING

```
JSON files
|
```



Next, the best lesson is `readStream` in detail: how Spark detects new JSON files, why streaming JSON needs a schema, and how to test it by adding files one by one.